

LLM Observability Platform

Project summary — what this is, what we built, and where it stands today

What this platform is

This is a self-built observability tool for LLM applications — a lighter, homegrown version of products like Braintrust or LangSmith. It captures every LLM call an application makes (the traces), tracks quality and cost over time, lets you evaluate prompts against a test suite, compare experiments, and now automatically checks every code change against that test suite before it merges.

It's built with a FastAPI + PostgreSQL backend and a React frontend, and it talks to three real LLM providers (Gemini, Groq, and OpenRouter) rather than mock data.

Where it started

The original version covered the basics: it recorded traces (a question in, an answer out), stored them with the model used and token counts, and had a simple dashboard, a Playground for trying prompts, and a Prompt Library. Cost figures were placeholders, not calculated from anything real.

The gap analysis

We watched a walkthrough of Braintrust — a well-known commercial LLM observability product — and compared it feature-by-feature against what this app already had. That produced a punch list of real gaps: no way to score answer quality automatically, no concept of an "experiment" (a batch of test runs you can compare over time), no nested view of multi-step calls, no way for a human to say "this answer was wrong," no history of how a prompt changed over time, no alerting, and no queue for flagging bad responses for review.

We deliberately did *not* build a few things that would need real new infrastructure this app doesn't have: a system for plugging in arbitrary custom AI providers, a SQL sandbox over trace data, and tool-calling observability. Those stayed as reference material rather than fake stand-ins.

Features built this session

Scorers

Automatic graders for an LLM's answer — for example, checking whether the answer actually addresses the question (relevance) and whether it sticks to the facts it was given (faithfulness), producing a real numeric score instead of a human having to read every response.

Experiments

A way to run a batch of test questions against the app, save the whole batch as a named "experiment," and then compare two experiments side-by-side — which questions got better, which got worse, and by how much. There's also an AI-generated written analysis of what changed between two experiments, so you don't have to eyeball a table of numbers to understand the story.

Nested span hierarchy

Real applications rarely make just one LLM call — there might be a retrieval step, then a reasoning step, then a final generation step. The Trace Details view now shows these as a proper nested tree (a "span" inside a "trace"), rather than one flat entry, so you can see the actual shape of a multi-step call.

Human feedback

Thumbs-up/thumbs-down buttons in the Playground let a person mark whether an answer was actually good, and that feedback is stored and shown alongside the trace — a second, human signal next to the automatic Scorer grades.

Prompt versioning

The Prompt Library now keeps a history every time a prompt is edited, instead of only ever showing the latest version — so you can see how a prompt evolved and what changed.

Alerts

Rules that watch metrics (like pass rate or average cost) and flag when something crosses a threshold you define, instead of requiring someone to keep checking a dashboard manually.

Review queue

Any trace can be flagged for human review, and flagged traces collect in a dedicated queue page — a workflow for "something looked off here, someone should double-check it" that didn't exist before.

Real bug fixes (not cosmetic)

Deprecated Gemini models and SDK. Google had shut down the Gemini models this app was calling, and the SDK being used was also end-of-life. Both were migrated to the currently-supported model family and SDK, so Gemini calls actually work again instead of silently failing.

Fabricated cost numbers. Cost & Usage was previously showing placeholder figures. It now calculates real cost from each provider's actual published per-token pricing and the real token counts from each call — an honest number, including an honest \$0 for the genuinely free-tier models, rather than a made-up one.

The CI Eval Gate

This was the largest single addition: an automated check that runs every time code is pushed or a pull request is opened, modeled on Braintrust's own open-source `eval-action`, but rebuilt from scratch to work against this app's own FastAPI backend instead of Braintrust's SDK.

What it does

- Spins up a real, temporary Postgres database and this app's backend inside GitHub's CI runner.
- Runs a small, checked-in set of test questions (currently 6) against the real backend and a real LLM provider (Groq).
- Compares the results — pass rate, faithfulness score, latency, cost — against a committed "baseline" file from the last known-good run.
- On a pull request, it posts the comparison as a comment directly on the PR, so anyone reviewing the change can see at a glance whether it made things better or worse, with no manual testing required.
- On a merge to the main branch, it updates that baseline file so the next PR compares against fresh numbers.

The debugging story, in plain terms

Getting this from "written" to "actually reliable" took real troubleshooting. The short version: everything worked correctly from a normal computer, but calls from inside GitHub's automated environment kept failing with a generic "connection error."

It turned out to have nothing to do with networking. The real cause, found in two separate rounds, was how the secret API key was pasted into GitHub's secret box — first with an invisible extra line break at the end, then with the surrounding quote marks accidentally included as part of the value. Both are invisible mistakes: a tool on your own computer quietly strips that kind of formatting for you, but GitHub's secret storage does not, so the key that looked fine locally was subtly different once it reached CI. Once the raw value was pasted in cleanly, everything passed immediately — the key itself had been valid all along.

Along the way we also fixed a real, separate gap: when a provider call failed, the error was being silently swallowed instead of shown anywhere — fixed so a failed call now reports what actually went wrong instead of just disappearing.

Fully verified, not just assumed

Before calling this done, we tested the whole pipeline for real: a genuine test pull request with a new test case was opened, and the gate correctly posted its results comment on that real PR. That PR was then merged the normal way. This confirmed every part of the pipeline actually works, not just the parts that had been tested before.

Housekeeping: because the API key was visible in a screenshot during debugging, it was rotated afterward (old key revoked, new one generated and re-added to both the local environment and the GitHub secret) as a security precaution — and the gate was re-verified end-to-end with the new key.

Where things stand today

Component	Status
Backend (FastAPI)	Running, verified against a real database
Database (Postgres)	Running, all migrations applied
Frontend (React/Vite)	Running, all pages wired up
CI Eval Gate	Green, verified via a real merged pull request
API key hygiene	Rotated and re-verified

Generated as a plain-language recap of the work completed in this project session.